NUL SMART LIBRARY SYSTEM

Course Name and Code: Data structures and Algorithms CS3400

Group Members:

- Mzwandile Hlapisi 202321287
- Rearabetsoe Monyako 202321246
- Teboho Montle 202407866
- Kabelo Lekhanya 202402333

Lecturer's Name: B. Ndaba(Mr.)

Submission Date: 19/10/2025

ABSTRACT

This mini-project presents a Smart Library System developed to manage book borrowing, returning, searching, and reservation in a university library. The main goal is to create a simple but efficient program that automates common library tasks and demonstrates the practical use of data structures in C++.

The system uses a combination of data structures to perform different operations effectively. A Binary Search Tree (BST) is used to organize books by title or author, allowing quick searching and sorting. A Hash Table stores book and user data, providing constant-time access for adding or finding records. Queues manage book reservations and waiting lists, ensuring users borrow books in a first-come, first-served order. A Linked List keeps track of the borrowing and returning history, making it easy to update records.

The system is built in **C++** using modular programming and object-oriented design. Each operation, such as borrowing or returning a book, is handled by a specific function or class. The overall outcome is a simple console-based library management system that demonstrates the power of using multiple data structures to solve a real-world problem efficiently.

INTRODUCTION

Problem Statement

Many university libraries still rely on manual record keeping or outdated systems to manage books. This often leads to missing records, long waiting times, and difficulty searching for available titles. Students may not know whether a book is borrowed or available, and librarians spend extra time checking and updating logs. There is a clear need for an automated system that can store, organize, and update book information quickly and accurately.

The Smart Library System was designed to solve these problems. It aims to provide an easy way for students and staff to borrow, return, and reserve books while maintaining accurate records. It also helps the library handle waiting lists automatically and reduces manual work.

Project Objectives

1. To design and implement a library system that allows book borrowing, returning, and searching.
2. To apply different data structures (BST, hash table, queue, linked list) to make the system efficient.
3. To allow quick searching of books by title or author.
4. To record users' borrowing history automatically.
5. To provide an easy-to-use console program for managing library operations.

Brief Description of the Theme

The Smart Library System combines data structures to handle different functions of a library. The BST helps organize and search for books efficiently. The Hash Table allows quick lookup of book and user details. Queues handle users waiting for books that are already borrowed, and Linked Lists store transaction records such as borrowing and returning activities. Each part of the system is carefully designed to demonstrate how data structures work together in solving real-life problems.

Scope and Limitations

This project focuses on managing printed books within a university library. It allows users to search, borrow, and return books, as well as join a waiting queue for unavailable titles. However, the system does not handle online services, e-books, or external database connections. All data is stored temporarily in memory during program execution. Future versions can include permanent file storage or a graphical user interface (GUI).

SYSTEM DESIGN

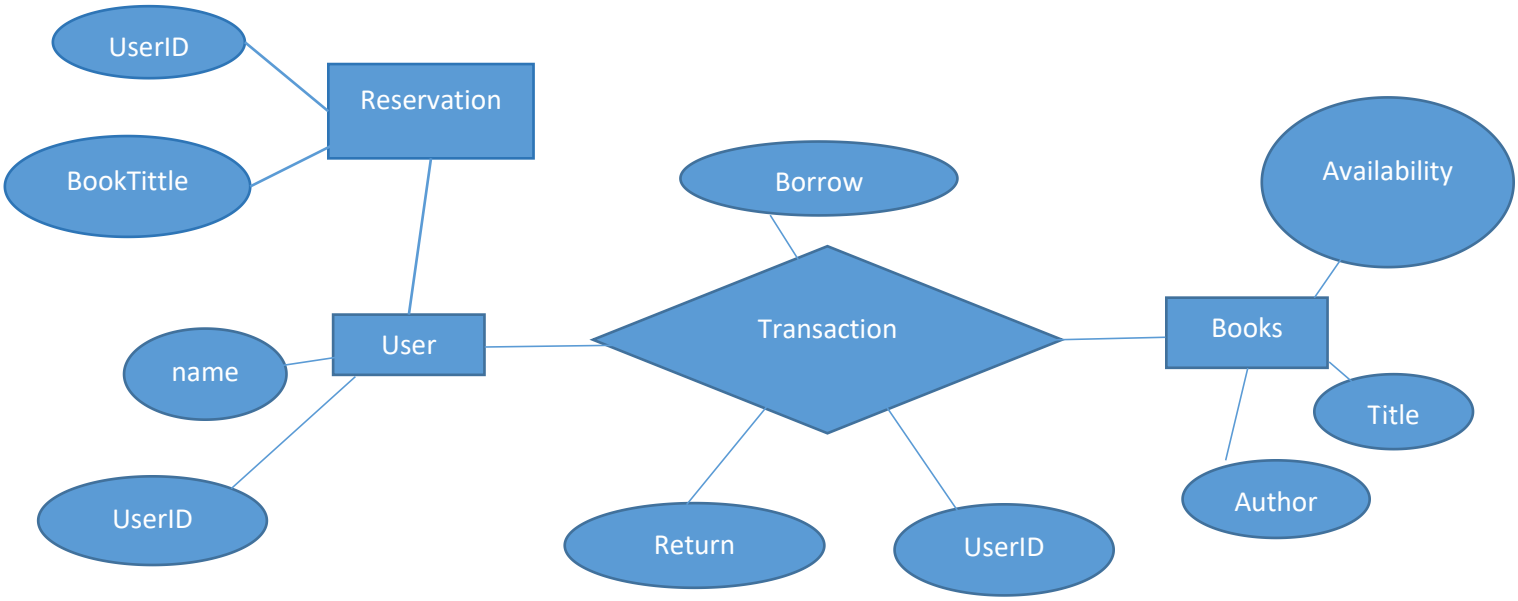
Overview

The system design describes how different modules and data structures work together to manage library operations. The Smart Library System follows a modular and layered structure, where each part of the program handles a specific task. This makes the program easier to understand, test, and maintain.

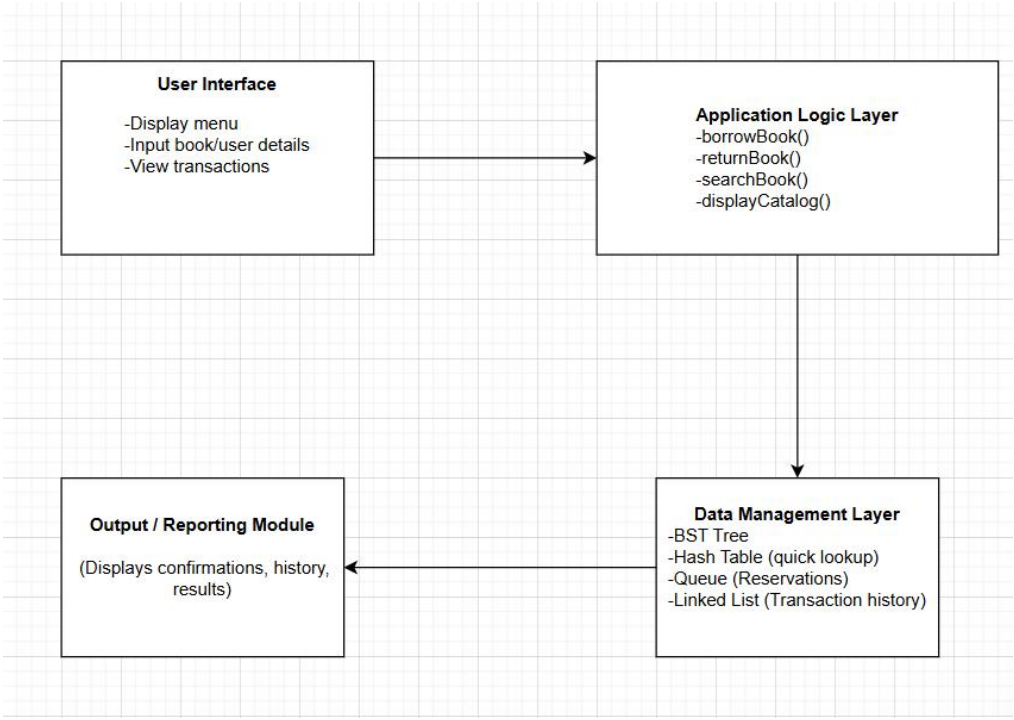
The system includes four main components:

1. User Interface Module: Handles user input and displays system messages.
2. Library Management Module: Controls book borrowing, returning, and reservation.
3. Data Structures Module: Implements BST, Hash Table, Queue, and Linked List.
4. Report Module: Displays records and search results.

Entity Relationship diagram



System Architecture



Choice of Data Structures

Feature	Data Structure	Justification
Search for books by title or author	Binary Search Tree (BST)	Allows fast searching, inserting, and sorting of books alphabetically.
Store user and book details	Hash Table	Enables constant-time access using book ID or user ID as keys.
Manage waiting list for borrowed books	Queue	Maintains the correct order(FIFO) for reservations.
Keep track of borrowing and returning history	Linked List	Makes it easy to add new transactions and update records.

Module Descriptions

1. **Book Management (BST & Hash Table)**
    - The system stores book details like title, author, and availability.
    - BST is used for organizing and searching books quickly.
    - Hash Table stores unique IDs for fast lookup.
2. **Reservation Handling (Queue)**
    - If a book is not available, the user is added to the reservation queue.
    - When the book is returned, the next user in the queue is notified.
3. **Transaction Log (Linked List)**
    - Every borrow or return operation is recorded as a node in a linked list.
    - The list shows borrowing history in the order transactions occurred.
4. **User Interaction**
    - The user interacts through simple menu options (e.g., Search, Borrow, Return).
    - The system displays messages or results using console output.

Sample Pseudocode for Borrow Operation

```
function borrowBook(userID, bookTitle):
    bookNode = BST.search(bookTitle)
    if bookNode == NULL:
        print "Book not found."
    else if bookNode.isAvailable:
        bookNode.isAvailable = false
        addToLinkedList(userID, bookTitle, "Borrowed")
        print "Book borrowed successfully."
```

```
    else:
        enqueue(waitingQueue[bookTitle], userID)
        print "Book already borrowed. You are added to the
waiting list."
```

Sample Pseudocode for Return Operation

```
function returnBook(userID, bookTitle):
    bookNode = BST.search(bookTitle)
    if bookNode == NULL:
        print "Book not found."
    else:
        bookNode.isAvailable = true
        addToLinkedList(userID, bookTitle, "Returned")
        if waitingQueue[bookTitle] is not empty:
            nextUser = dequeue(waitingQueue[bookTitle])
            print "Book now available for " + nextUser
```

Design Summary

The Smart Library System uses a combination of data structures to perform library operations efficiently. The BST provides organized storage for books, the Hash Table allows quick lookups, the Queue ensures fair waiting order, and the Linked List keeps a clear record of transactions. Together, these structures make the system modular, efficient, and practical for real-world use.